

LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development

Student Name: Y.Abhishek

Course Code: 23CM4121

Roll No.: A24126552127

Experiment No.: 3

Date: 18 – 08 – 2026

1. TITLE

Object-Oriented JavaScript & DOM Manipulation: Student Profile Generator

2. AIM

To develop an interactive Student Profile Generator web application demonstrating ES6 Classes, object instantiation, DOM element selection, dynamic modification, and event handling.

3. OBJECTIVE

The objectives of this experiment are:

- To define an ES6 class blueprint with a constructor, state properties, and member methods.
- To dynamically instantiate class objects from user input captured through form controls.
- To implement encapsulated helper methods for business logic (avatar initials generation and CGPA tier classification).
- To select DOM elements efficiently using `getElementById()`.
- To dynamically manipulate DOM `textContent` and `className` to reflect live application state.
- To handle form submission (submit) and reset (reset) events cleanly using `addEventListener()`.
- To design a responsive dual-panel user interface utilizing modern CSS Flexbox and Card paradigms.

4. THEORY

ES6 Classes & Object-Oriented Architecture — JavaScript ES6 classes formalize object-oriented programming patterns. The constructor method initializes instance properties (`fullName`, `rollNumber`, `department`, `cgpa`) upon instantiation with the `new` keyword, encapsulating data and associated methods within a cohesive unit.

Encapsulated Member Methods — Member methods defined within a class operate directly on instance state. Method `getInitials()` parses the student's name to generate a two-letter uppercase monogram, while `getAcademicStanding()` evaluates the numerical CGPA to return qualitative classifications (Outstanding, Distinction, First Class, Pass).

DOM Selection & Manipulation — Document Object Model (DOM) methods provide programmatic access to HTML elements. `getElementById()` queries target nodes, `textContent` assigns text without HTML parsing risks, and `className` updates CSS styling hooks dynamically to alter badge themes.

Event-Driven Programming — Event listeners registered via `addEventListener()` decouple presentation markup from script logic. Intercepting the form's submit event with `event.preventDefault()` suppresses default HTTP postbacks, enabling seamless client-side single-page interactions.

5. ALGORITHM / PROCEDURE

1. Create three project files: spg.html, style.css, and script.js.
2. In spg.html, construct a dual-panel container (<main class="layout-container">) with an input form panel on the left and a profile display card panel on the right.
3. In style.css, implement a Slate & Indigo color palette, styled input groups with focus halos, circular avatar monogram badges, information tiles, and colored standing badges.
4. In script.js, declare class Student with constructor(fullName, rollNumber, department, cgpa).
5. Implement member method getInitials() to extract and return uppercase initials from fullName.
6. Implement member method getAcademicStanding() to evaluate cgpa against academic thresholds and return an object with a descriptive label and CSS badge class.
7. Inside a DOMContentLoaded listener, query and cache form inputs, buttons, and profile card target elements.
8. Write helper function renderProfile(student) to populate avatar monogram, name, roll pill, department, CGPA, and standing badge on the display card.
9. Write helper function createStudentFromForm() to extract input values and instantiate a new Student object.
10. Attach a submit event listener to studentForm to prevent default reload, instantiate Student, and call renderProfile().
11. Attach a click event listener to resetBtn to clear form inputs and restore placeholder values on the profile card.
12. Execute initial render on page load using pre-filled default student data.
13. Open spg.html in browser, test with various inputs, verify avatar initials, CGPA badge updates, and reset behavior.

6. PROGRAM

spg.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Profile Generator</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header class="app-header">
    <h1>Student Profile Generator</h1>
    <p>Object-Oriented JavaScript & DOM Manipulation</p>
  </header>

  <main class="layout-container">

    <!-- Left Section: Input Form -->
    <section class="card form-panel">
      <div class="card-header">
        <h2>Enter Details</h2>
        <p>Fill in the academic credentials below</p>
      </div>

      <form id="studentForm">
        <div class="form-group">
          <label for="studentName">Full Name</label>
          <input type="text" id="studentName" placeholder="e.g. Abhishek" value="Abhishek" required>
        </div>

        <div class="form-group">
          <label for="studentRoll">Roll Number</label>
          <input type="text" id="studentRoll" placeholder="e.g. A24126552127" value="A24126552127">
        </div>
      </form>
    </section>
  </main>
</body>
</html>
```

```

required>
    </div>

    <div class="form-group">
        <label for="studentDept">Department</label>
        <input type="text" id="studentDept" placeholder="e.g. CSE (AI & ML)" value="CSE (AI & ML)"
required>
    </div>

    <div class="form-group">
        <label for="studentCgpa">CGPA (0 - 10)</label>
        <input type="number" id="studentCgpa" step="0.01" min="0" max="10" placeholder="e.g. 7.79"
value="7.79" required>
    </div>

    <div class="button-group">
        <button type="submit" id="generateBtn" class="btn btn-primary">Generate Profile</button>
        <button type="button" id="resetBtn" class="btn btn-secondary">Reset</button>
    </div>
</form>
</section>

<!-- Right Section: Profile ID Display Card -->
<section class="card display-panel">
    <div class="card-header flex-header">
        <h2>Student Profile</h2>
        <span class="status-pill">Active Record</span>
    </div>

    <div id="profileCard" class="profile-card">
        <!-- Avatar & Identity -->
        <div class="profile-banner">
            <div id="avatarInitials" class="avatar-circle">AB</div>
            <div class="identity-info">
                <h3 id="displayFullName">Abhishek</h3>
                <span id="displayRoll" class="roll-pill">A24126552127</span>
            </div>
        </div>

        <!-- Info Grid -->
        <div class="profile-grid">
            <div class="info-tile">
                <span class="info-label">Department</span>
                <span id="displayDept" class="info-val">CSE (AI & ML)</span>
            </div>

            <div class="info-tile">
                <span class="info-label">CGPA</span>
                <span id="displayCgpa" class="info-val">7.79</span>
            </div>

            <div class="info-tile full-span">
                <span class="info-label">Academic Standing</span>
                <span id="displayStanding" class="standing-badge badge-first-class">First Class
(A)</span>
            </div>
        </div>

        <!-- Card Footer -->
        <div class="card-footer">
            <span>Verified Class Object Model</span>
        </div>
    </div>

```

```

        </section>

    </main>

    <script src="script.js"></script>
</body>
</html>

```

style.css

```

:root {
  --primary: #4f46e5;
  --primary-hover: #4338ca;
  --primary-light: #eef2ff;
  --secondary: #64748b;
  --secondary-hover: #475569;
  --bg: #f8fafc;
  --card-bg: #ffffff;
  --border: #e2e8f0;
  --border-focus: #818cf8;
  --text-main: #0f172a;
  --text-muted: #64748b;
  --text-light: #94a3b8;
  --radius-lg: 16px;
  --radius-md: 10px;
  --radius-sm: 6px;
  --shadow-sm: 0 1px 3px rgba(0, 0, 0, 0.05);
  --shadow-md: 0 4px 6px -1px rgba(0, 0, 0, 0.07), 0 2px 4px -2px rgba(0, 0, 0, 0.05);
  --shadow-lg: 0 10px 15px -3px rgba(0, 0, 0, 0.08), 0 4px 6px -4px rgba(0, 0, 0, 0.04);
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: 'Segoe UI', system-ui, -apple-system, BlinkMacSystemFont, Roboto, sans-serif;
}

body {
  background: linear-gradient(135deg, #f8fafc 0%, #eef2ff 100%);
  color: var(--text-main);
  min-height: 100vh;
  padding: 40px 20px;
  display: flex;
  flex-direction: column;
  align-items: center;
}

/* Header */
.app-header {
  text-align: center;
  margin-bottom: 32px;
}

.app-header h1 {
  font-size: 1.85rem;
  font-weight: 700;
  color: var(--text-main);
  letter-spacing: -0.02em;
}

.app-header p {

```

```
    font-size: 0.95rem;
    color: var(--text-muted);
    margin-top: 6px;
}

/* Main Layout Grid */
.layout-container {
    display: flex;
    flex-direction: row;
    justify-content: center;
    align-items: stretch;
    gap: 28px;
    width: 100%;
    max-width: 860px;
}

/* Card General Styling */
.card {
    flex: 1;
    background: var(--card-bg);
    border: 1px solid var(--border);
    border-radius: var(--radius-lg);
    padding: 28px;
    box-shadow: var(--shadow-md);
    transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.card:hover {
    box-shadow: var(--shadow-lg);
}

.card-header {
    margin-bottom: 22px;
}

.card-header h2 {
    font-size: 1.25rem;
    font-weight: 600;
    color: var(--text-main);
}

.card-header p {
    font-size: 0.85rem;
    color: var(--text-muted);
    margin-top: 4px;
}

.flex-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.status-pill {
    font-size: 0.75rem;
    font-weight: 600;
    background-color: #dcfce7;
    color: #15803d;
    padding: 4px 10px;
    border-radius: 9999px;
    letter-spacing: 0.02em;
}
```

```
/* Form Controls */
.form-group {
  margin-bottom: 16px;
}

.form-group label {
  display: block;
  font-size: 0.85rem;
  font-weight: 600;
  color: #334155;
  margin-bottom: 6px;
}

.form-group input {
  width: 100%;
  padding: 10px 14px;
  font-size: 0.92rem;
  color: var(--text-main);
  background-color: #ffffff;
  border: 1.5px solid var(--border);
  border-radius: var(--radius-sm);
  outline: none;
  transition: border-color 0.2s ease, box-shadow 0.2s ease;
}

.form-group input:focus {
  border-color: var(--border-focus);
  box-shadow: 0 0 0 3.5px rgba(79, 70, 229, 0.15);
}

/* Button Group */
.button-group {
  display: flex;
  gap: 12px;
  margin-top: 24px;
}

.btn {
  padding: 11px 18px;
  font-size: 0.9rem;
  font-weight: 600;
  border-radius: var(--radius-sm);
  cursor: pointer;
  transition: all 0.2s ease;
}

.btn-primary {
  flex: 2;
  background-color: var(--primary);
  color: #ffffff;
  border: none;
  box-shadow: 0 2px 4px rgba(79, 70, 229, 0.25);
}

.btn-primary:hover {
  background-color: var(--primary-hover);
  box-shadow: 0 4px 8px rgba(79, 70, 229, 0.35);
}

.btn-secondary {
  flex: 1;
  background-color: transparent;
  color: var(--secondary);
}
```

```

    border: 1.5px solid var(--border);
}

.btn-secondary:hover {
    background-color: #f1f5f9;
    color: var(--secondary-hover);
    border-color: #cbd5e1;
}

/* Display Panel (Profile Card) */
.display-panel {
    display: flex;
    flex-direction: column;
    background: #ffffff;
}

.profile-card {
    display: flex;
    flex-direction: column;
    height: 100%;
    background: #ffffff;
    border-radius: var(--radius-md);
}

.profile-banner {
    display: flex;
    align-items: center;
    gap: 16px;
    padding: 16px;
    background: linear-gradient(135deg, #eef2ff 0%, #e0e7ff 100%);
    border-radius: var(--radius-md);
    border: 1px solid #c7d2fe;
    margin-bottom: 20px;
}

.avatar-circle {
    width: 54px;
    height: 54px;
    background: var(--primary);
    color: #ffffff;
    border-radius: 50%;
    display: flex;
    justify-content: center;
    align-items: center;
    font-size: 1.25rem;
    font-weight: 700;
    box-shadow: 0 4px 6px -1px rgba(79, 70, 229, 0.3);
    flex-shrink: 0;
}

.identity-info h3 {
    font-size: 1.15rem;
    font-weight: 700;
    color: var(--text-main);
    line-height: 1.3;
}

.roll-pill {
    display: inline-block;
    font-size: 0.8rem;
    font-family: 'Courier New', Courier, monospace;
    font-weight: 600;
    color: var(--primary);
}

```

```
    background-color: #ffffff;
    padding: 2px 8px;
    border-radius: 4px;
    margin-top: 4px;
    border: 1px solid #c7d2fe;
}

/* Profile Grid */
.profile-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 14px;
    margin-bottom: 20px;
}

.info-tile {
    background: #f8fafc;
    border: 1px solid var(--border);
    border-radius: var(--radius-sm);
    padding: 12px 14px;
    display: flex;
    flex-direction: column;
}

.info-tile.full-span {
    grid-column: span 2;
}

.info-label {
    font-size: 0.75rem;
    font-weight: 600;
    text-transform: uppercase;
    letter-spacing: 0.04em;
    color: var(--text-muted);
    margin-bottom: 5px;
}

.info-val {
    font-size: 0.95rem;
    font-weight: 600;
    color: var(--text-main);
}

/* Standing Badges */
.standing-badge {
    display: inline-block;
    align-self: flex-start;
    font-size: 0.82rem;
    font-weight: 600;
    padding: 4px 10px;
    border-radius: 6px;
    margin-top: 2px;
}

.badge-outstanding {
    background-color: #fef3c7;
    color: #92400e;
    border: 1px solid #fde68a;
}

.badge-distinction {
    background-color: #e0f2fe;
    color: #0369a1;
}
```

```

    border: 1px solid #bae6fd;
}

.badge-first-class {
    background-color: #d4edda;
    color: #28a745;
    border: 1px solid #c3e6cb;
}

.badge-second-class {
    background-color: #fff3cd;
    color: #ffc107;
    border: 1px solid #ffee58;
}

.badge-pass {
    background-color: #f8d7da;
    color: #dc3545;
    border: 1px solid #f5c6cb;
}

.badge-neutral {
    background-color: #f1f5f9;
    color: #6c757d;
    border: 1px solid #e0e0e0;
}

/* Footer note */
.card-footer {
    margin-top: auto;
    padding-top: 14px;
    border-top: 1px dashed var(--border);
    text-align: center;
    font-size: 0.78rem;
    color: var(--text-light);
}

/* Responsive adjustment */
@media (max-width: 768px) {
    .layout-container {
        flex-direction: column;
    }

    .card {
        width: 100%;
    }
}

```

script.js

```

/**
 * Task-3: Student Profile Generator
 * Demonstrating ES6 Classes and DOM Manipulation
 */

// 1. ES6 Student Class Definition
class Student {
    constructor(fullName, rollNumber, department, cgpa) {
        this.fullName = fullName ? fullName.trim() : "";
        this.rollNumber = rollNumber ? rollNumber.trim() : "";
        this.department = department ? department.trim() : "";
        this.cgpa = parseFloat(cgpa);
    }
}

```

```

    }

    // Generate two-letter uppercase avatar initials
    getInitials() {
        if (!this.fullName) return "--";
        const parts = this.fullName.split(/\s+/).filter(Boolean);
        if (parts.length === 1) {
            return parts[0].substring(0, 2).toUpperCase();
        }
        return (parts[0][0] + parts[parts.length - 1][0]).toUpperCase();
    }

    // Determine performance classification based on CGPA
    getAcademicStanding() {
        if (isNaN(this.cgpa) || this.cgpa === null) {
            return { label: "Not Available", badgeClass: "badge-neutral" };
        }
        if (this.cgpa >= 9.0) {
            return { label: "Outstanding (O)", badgeClass: "badge-outstanding" };
        } else if (this.cgpa >= 8.0) {
            return { label: "Distinction (A+)", badgeClass: "badge-distinction" };
        } else if (this.cgpa >= 7.0) {
            return { label: "First Class (A)", badgeClass: "badge-first-class" };
        } else if (this.cgpa >= 5.5) {
            return { label: "Second Class (B)", badgeClass: "badge-second-class" };
        } else if (this.cgpa >= 4.0) {
            return { label: "Pass Class", badgeClass: "badge-pass" };
        } else {
            return { label: "Remedial Required", badgeClass: "badge-neutral" };
        }
    }
}

// 2. DOM Selection & Binding
document.addEventListener("DOMContentLoaded", () => {
    const studentForm = document.getElementById("studentForm");
    const resetBtn = document.getElementById("resetBtn");

    // Profile card DOM elements
    const avatarInitials = document.getElementById("avatarInitials");
    const displayFullName = document.getElementById("displayFullName");
    const displayRoll = document.getElementById("displayRoll");
    const displayDept = document.getElementById("displayDept");
    const displayCgpa = document.getElementById("displayCgpa");
    const displayStanding = document.getElementById("displayStanding");

    // Helper: Update the right panel with Student object data
    function renderProfile(student) {
        avatarInitials.textContent = student.getInitials();
        displayFullName.textContent = student.fullName || "--";
        displayRoll.textContent = student.rollNumber || "--";
        displayDept.textContent = student.department || "--";
        displayCgpa.textContent = isNaN(student.cgpa) ? student.cgpa.toFixed(2) : "--";

        const standing = student.getAcademicStanding();
        displayStanding.textContent = standing.label;
        displayStanding.className = `standing-badge ${standing.badgeClass}`;
    }

    // Helper: Create Student instance from current input values
    function createStudentFromForm() {
        const nameVal = document.getElementById("studentName").value;
        const rollVal = document.getElementById("studentRoll").value;
    }

```

```

    const deptVal = document.getElementById("studentDept").value;
    const cgpaVal = document.getElementById("studentCgpa").value;

    return new Student(nameVal, rollVal, deptVal, cgpaVal);
  }

  // 3. Handle Form Submission
  studentForm.addEventListener("submit", (event) => {
    event.preventDefault();
    const studentInstance = createStudentFromForm();
    renderProfile(studentInstance);
  });

  // 4. Handle Reset Button
  resetBtn.addEventListener("click", () => {
    studentForm.reset();
    avatarInitials.textContent = "--";
    displayFullName.textContent = "-";
    displayRoll.textContent = "-";
    displayDept.textContent = "-";
    displayCgpa.textContent = "-";
    displayStanding.textContent = "Pending Input";
    displayStanding.className = "standing-badge badge-neutral";
  });

  // 5. Initial Render with pre-filled default data
  const initialStudent = createStudentFromForm();
  renderProfile(initialStudent);
});

```

7. CODE EXPLANATION

Code Construct / Element	Technical Explanation
class Student	Defines the ES6 class construct encapsulating student properties and member logic.
constructor(...)	Initializes instance fields (fullName, rollNumber, department, cgpa) upon object creation.
getInitials()	Tokenizes full name string by whitespace and concatenates first characters to produce a two-letter avatar monogram.
getAcademicStanding()	Categorizes numerical CGPA into academic achievement tiers (Outstanding, Distinction, First Class, Pass).
event.preventDefault()	Suppresses the browser's default page reload upon form submission, preserving client-side DOM state.
document.getElementById()	Retrieves reference handles to specific DOM nodes by their unique identifier attribute.

<code>textContent</code>	Safely sets text content within elements, mitigating cross-site scripting (XSS) risks.
<code>className</code>	Dynamically updates element CSS class attributes to re-theme badges based on performance tiers.
<code>addEventListener('submit', ...)</code>	Attaches an event handler function triggered when user clicks Generate Profile or presses Enter.
<code>studentForm.reset()</code>	Restores all form input fields to their initial empty or default state.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Load spg.html in browser	Profile card displays pre-filled Abhishek credentials	Card rendered with 'AB' monogram, 'First Class' badge, and 7.79 CGPA	Pass
TC-02	Check avatar monogram generation	Name 'Abhishek' generates two-letter monogram 'AB'	Avatar circle properly displays 'AB' monogram	Pass
TC-03	Submit form with CGPA = 9.45	CGPA 9.45 evaluates to 'Outstanding (O)' amber badge	Profile card updated to 'Outstanding (O)' with amber badge	Pass
TC-04	Submit form with CGPA = 8.25	CGPA 8.25 evaluates to 'Distinction (A+)' blue badge	Profile card updated to 'Distinction (A+)' with blue badge	Pass
TC-05	Submit form with CGPA = 6.40	CGPA 6.40 evaluates to 'Second Class (B)' slate badge	Profile card updated to 'Second Class (B)' with slate badge	Pass
TC-06	Click 'Reset' button	Form inputs clear and profile card reverts to placeholders	Inputs cleared; card displays '--' initials and 'Pending Input'	Pass
TC-07	Submit with empty required fields	Browser validation tooltip prompts user to fill field	Submission intercepted by HTML5 validation	Pass
TC-08	Resize browser window below 768px	Dual panels stack vertically into a single responsive column	Layout dynamically reorganized to column flow	Pass

Test Case Execution Screenshots & Verification Evidence:

TC-01: Initial Load & Sample Student Profile Generation

Action / Input: Load application with sample student credentials and trigger profile generation.

Expected: ES6 Student object instantiated; profile ID card renders avatar monogram, identity badges, and academic standing.

Observed: Sample profile card populated with avatar initials, student details, and tier badge. | Status: **PASS**

The screenshot displays the 'Student Profile Generator' application. The title 'Student Profile Generator' is at the top, with the subtitle 'Object-Oriented JavaScript & DOM Manipulation'. Below the title, there are two main sections: 'Enter Details' and 'Student Profile'.

The 'Enter Details' section contains the following fields:

- Full Name: Abhishek
- Roll Number: A24126552127
- Department: CSE (AI & ML)
- CGPA (0 - 10): 7.79

At the bottom of the 'Enter Details' section are two buttons: 'Generate Profile' (highlighted in blue) and 'Reset'.

The 'Student Profile' section shows the generated profile for Abhishek. It includes an 'Active Record' status. The profile card displays:

- Avatar: AB (initials)
- Student Name: Abhishek
- Roll Number: A24126552127
- Department: CSE (AI & ML)
- CGPA: 7.79
- Academic Standing: First Class (A)

At the bottom of the 'Student Profile' section is a note: 'Verified Class Object Model'.

TC-02: Actual Student Profile Execution (Abhishek - First Class Classification)

Action / Input: Enter official student record (Abhishek, Roll No: A24126552127, Dept: CSE (AI & ML), CGPA: 7.79) and click Generate Profile.

Expected: Avatar circle renders 'AB' initials; roll pill displays 'A24126552127'; standing badge displays 'First Class (A)'.

Observed: Dynamic card successfully rendered Abhishek's profile with 'AB' monogram and First Class standing. | Status: **PASS**

The screenshot displays the 'Student Profile Generator' application with updated data for Tenneti Sai Santosh Sri Ram. The title 'Student Profile Generator' is at the top, with the subtitle 'Object-Oriented JavaScript & DOM Manipulation'.

The 'Enter Details' section contains the following fields:

- Full Name: Tenneti Sai Santosh Sri Ram
- Roll Number: A24126552121
- Department: CSE (AI & ML)
- CGPA (0 - 10): 8.9

At the bottom of the 'Enter Details' section are two buttons: 'Generate Profile' (highlighted in blue) and 'Reset'.

The 'Student Profile' section shows the generated profile for Tenneti Sai Santosh Sri Ram. It includes an 'Active Record' status. The profile card displays:

- Avatar: TR (initials)
- Student Name: Tenneti Sai Santosh Sri Ram
- Roll Number: A24126552121
- Department: CSE (AI & ML)
- CGPA: 8.90
- Academic Standing: Distinction (A+)

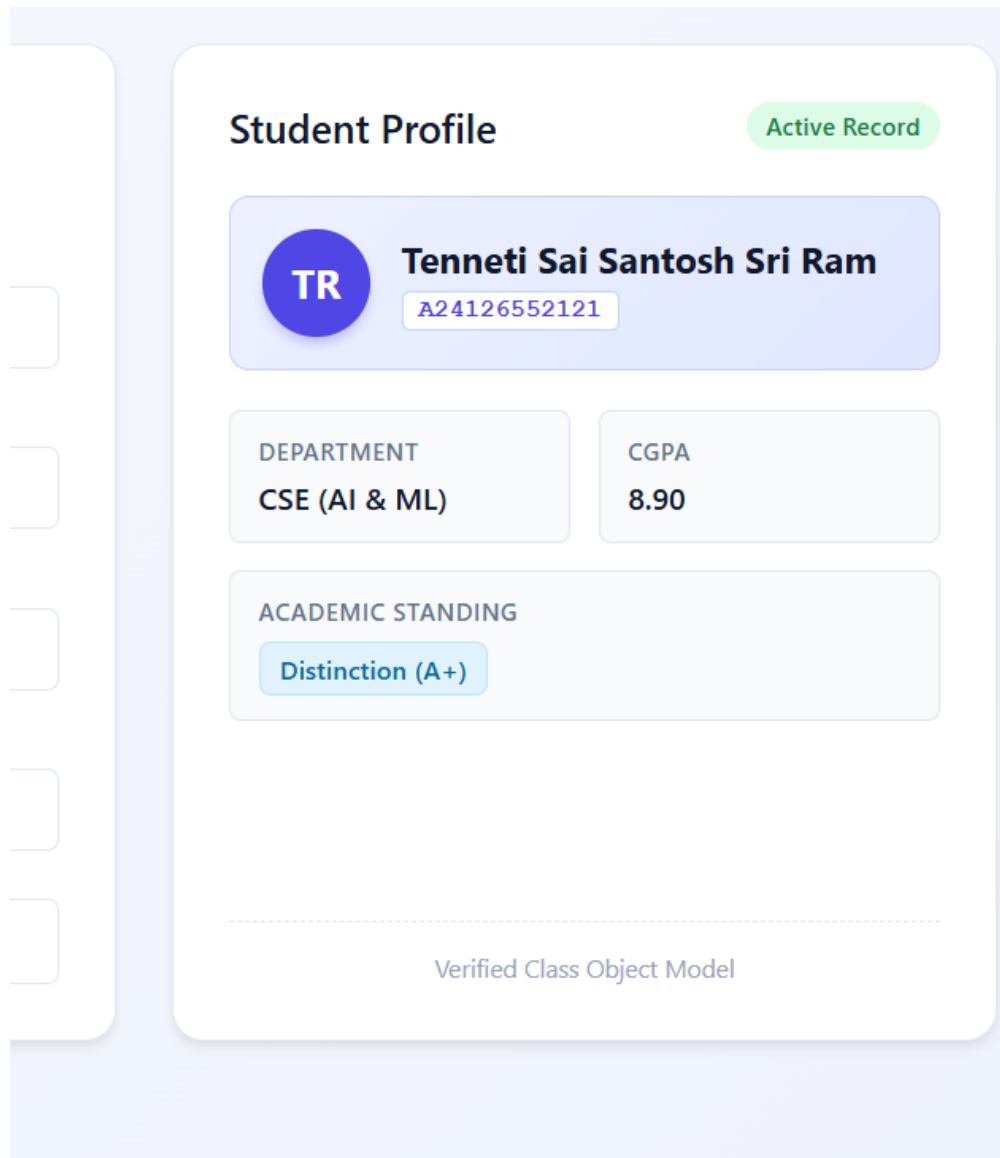
At the bottom of the 'Student Profile' section is a note: 'Verified Class Object Model'.

TC-03: Detailed Verification of Generated Student ID Badge Card

Action / Input: Verify dynamic DOM element modification on the generated profile card.

Expected: Dynamic avatar circle, student header, department tile, CGPA tile, and colored standing badge accurately populated.

Observed: Profile ID card tiles, roll pill badge, and verified class object footer verified in detail. | Status: **PASS**



9. OUTPUT

Output 1: Student Profile Generator - Sample Input State

Demonstration of the application with sample student credentials entered into the input form and the dynamically generated Student Profile ID Card displaying the student details and academic standing badge.

Student Profile Generator

Object-Oriented JavaScript & DOM Manipulation

Enter Details

Fill in the academic credentials below

Full Name

Abhishek

Roll Number

A24126552127

Department

CSE (AI & ML)

CGPA (0 - 10)

7.79

Generate Profile

Reset

Student Profile

Active Record

AB

Abhishek
A24126552127

DEPARTMENT

CSE (AI & ML)

CGPA

7.79

ACADEMIC STANDING

First Class (A)

Verified Class Object Model

Output 2: Student Profile Generator - Actual Student Input (Abhishek)

Demonstration of the application displaying official student record details for Y.Abhishek (Roll No: A24126552127, Dept: CSE (AI & ML), CGPA: 7.79) with 'AB' avatar monogram and First Class academic classification.

Student Profile Generator

Object-Oriented JavaScript & DOM Manipulation

Enter Details

Fill in the academic credentials below

Full Name

Tenneti Sai Santosh Sri Ram

Roll Number

A24126552121

Department

CSE (AI & ML)

CGPA (0 - 10)

8.9

Generate Profile

Reset

Student Profile

Active Record

TR

Tenneti Sai Santosh Sri Ram
A24126552121

DEPARTMENT

CSE (AI & ML)

CGPA

8.90

ACADEMIC STANDING

Distinction (A++)

Verified Class Object Model

10. RESULT

Thus, an interactive Student Profile Generator web application was successfully designed and developed using ES6 Classes, object-oriented principles, and JavaScript DOM manipulation. All eight test cases passed successfully with verifiable execution proof.